



NOMBRE DEL ESTUDIANTE:

Francisco Xavier Gil Ginez

DOCENTE:

José Miguel Carrera Pacheco

MATERIA:

Desarrollo Web Profesional

CUATRIMESTRE ENERO-ABRIL

2026

PROTECCIÓN DE RUTAS Y RECURSOS POR ROL

PARTE I — INVESTIGACIÓN TEÓRICA

1. ¿Qué es un Middleware?

Un middleware es una función que se ejecuta en el ciclo de solicitud-respuesta de una aplicación web, entre el momento en que el servidor recibe la petición y el momento en que envía la respuesta. En el contexto de frameworks como Express.js, un middleware tiene acceso al objeto de solicitud (req), al objeto de respuesta (res) y a la función next(), que permite pasar el control al siguiente middleware de la cadena.

La arquitectura de middleware funciona como una tubería (pipeline): cada petición HTTP atraviesa una serie de funciones en orden. Si alguna función detecta un problema (por ejemplo, que el token JWT es inválido), puede cortar el flujo y responder inmediatamente sin llegar al controlador final.

1.1 Estructura de un Middleware en Express.js

```
function miMiddleware(req, res, next) {  
  // 1. Leer datos de la solicitud (cabeceras, cuerpo, etc.)  
  // 2. Tomar una decisión (autorizar, rechazar, transformar)  
  // 3a. Continuar: llamar a next()  
  // 3b. Cortar: responder con res.status(4xx).json(...)  
}
```

1.2 Tipos de Middleware

Tipo	Descripción y uso típico
Middleware de aplicación	Se registra con app.use() y aplica a todas las rutas. Ejemplo: helmet(), cors(), express.json().
Middleware de ruta	Se aplica a rutas específicas. Ejemplo: authenticate solo en rutas privadas.
Middleware de error	Tiene cuatro parámetros (err, req, res, next). Captura errores no manejados.

Middleware de terceros	Librería externa que extiende funcionalidad. Ejemplo: morgan (logging), rate-limiter.
Middleware incorporado	Viene con Express. Ejemplo: <code>express.json()</code> , <code>express.static()</code> .

1.3 El Ciclo Solicitud-Respuesta con Middleware

Cuando un cliente hace una petición al servidor, el flujo sigue este orden:

1. El cliente envía una petición HTTP (GET, POST, etc.) con cabeceras y cuerpo.
2. Express recorre la cadena de middlewares en el orden en que fueron registrados.
3. Cada middleware puede modificar req/res, llamar a `next()` o enviar una respuesta.
4. Si ningún middleware interrumpe el flujo, la petición llega al controlador final.
5. El controlador procesa la lógica de negocio y envía la respuesta al cliente.

2. ¿Cómo se Protegen Rutas en el Backend?

Proteger una ruta significa garantizar que solo usuarios autorizados puedan acceder a sus recursos. En aplicaciones RESTful, el mecanismo más común es la autenticación basada en tokens JWT (JSON Web Tokens) combinada con control de acceso por rol (RBAC). La protección opera en tres niveles complementarios:

2.1 Autenticación con JWT

JWT (JSON Web Token) es un estándar abierto (RFC 7519) que define una forma compacta y segura de transmitir información entre partes como un objeto JSON firmado digitalmente. Un JWT tiene tres partes separadas por puntos: Header.Payload.Signature.

Parte del JWT	Contenido
Header	Tipo de token (JWT) y algoritmo de firma (HS256, RS256).
Payload (Claims)	Datos del usuario: id, role, zone_id, sid (session ID), iat, exp.
Signature	HMAC-SHA256 del Header + Payload firmado con el secreto del servidor.

El flujo de autenticación funciona así: el usuario inicia sesión con credenciales válidas; el servidor genera un access token (corto, ~15 min) y un refresh token (largo, días); el cliente almacena los tokens y los envía en la cabecera `Authorization: Bearer <token>` en cada petición; el servidor valida la firma del token y extrae el payload sin consultar la base de datos en cada request.

2.2 Validación de Sesión Activa en Base de Datos

Una mejora importante sobre la autenticación JWT pura es la validación de sesión en base de datos. Aunque el JWT sea válido criptográficamente, si el servidor revocó esa sesión (por cierre de sesión, sesión expirada o política de seguridad), la petición debe rechazarse. Esto se implementa almacenando un identificador de sesión (sid) en el token y verificando en cada petición que ese sid siga activo en la tabla `user_sessions`.

2.3 Autorización por Rol (RBAC)

Role-Based Access Control (RBAC) es un modelo de control de acceso en el que los permisos se asignan a roles, y los roles se asignan a usuarios. Este modelo facilita la gestión de permisos a escala porque en lugar de configurar permisos individuales por usuario, se administran los roles. En el sistema analizado existen tres roles:

Rol	Descripción	Permisos principales
admin	Administrador del sistema	Gestión de usuarios, auditoría, todas las rutas.
guard	Guardia de seguridad	Registrar eventos de auditoría.
driver	Conductor	Acceso solo a sus propios datos.

2.4 Rutas Públicas vs Rutas Protegidas

No todas las rutas deben protegerse. La regla de diseño es: las rutas que generan o validan credenciales son públicas; todo lo demás requiere autenticación y posiblemente autorización por rol.

Tipo de Ruta	Ejemplos en el sistema
Públicas (sin middleware)	POST <code>/api/auth/login</code> , POST <code>/api/auth/register</code> , POST <code>/api/auth/recover</code>
Autenticadas (solo JWT)	GET <code>/api/auth/sessions</code> , POST <code>/api/auth/logout</code> , GET <code>/api/auth/me</code>
Autenticadas + Rol admin	GET <code>/api/admin/users</code>
Autenticadas + Rol guard/admin	POST <code>/api/auditoria/registrar</code>

3. Errores de Seguridad Comunes en el Backend

Los errores de seguridad en APIs REST son frecuentes y tienen consecuencias graves. Conocerlos es el primer paso para evitarlos. La organización OWASP (Open Web

Application Security Project) publica anualmente una lista de los riesgos más críticos; los siguientes corresponden a los más relevantes para APIs:

Error de Seguridad	Descripción	Consecuencia
Autenticación rota	No verificar el JWT o hacerlo incorrectamente (por ejemplo, aceptar algoritmo 'none').	Cualquier usuario puede falsificar su identidad.
Autorización excesivamente permisiva	No verificar el rol o verificarlo solo en el frontend.	Usuarios con roles bajos acceden a recursos de administración.
Exposición de datos sensibles	Enviar password_hash, tokens internos o datos personales en respuestas de API.	Fuga de información que puede usarse para ataques.
Tokens sin expiración	Access tokens con vida larga o sin validación de revocación.	Un token robado puede usarse indefinidamente.
Inyección SQL	Construir queries con string concatenation en lugar de parámetros.	Extracción o destrucción total de la base de datos.
Sin limitación de tasa	Permitir ilimitadas solicitudes por IP o usuario.	Ataques de fuerza bruta o denegación de servicio.
CORS abierto	Configurar Access-Control-Allow-Origin: * en producción.	Cualquier sitio web puede hacer peticiones autenticadas.
Headers HTTP inseguros	No usar Helmet u equivalente.	Vulnerabilidades XSS, clickjacking, sniffing de contenido.
Sin validación de entrada	Confiar en los datos que envía el cliente sin sanitizarlos.	Inyección de código, overflow, comportamientos inesperados.
Información en mensajes de error	Devolver stack traces o mensajes de BD en respuestas de error.	El atacante obtiene información sobre la arquitectura interna.

4. ¿Cómo se Valida el Acceso por Rol?

La validación de acceso por rol es un proceso de dos pasos que debe ocurrir en el servidor en cada petición a un recurso protegido. Es un error grave implementar esta validación solo en el cliente (frontend), ya que cualquier usuario puede manipular el cliente.

4.1 Proceso de Validación

6. Autenticación: verificar que el token JWT sea válido y no haya expirado.
7. Extracción de identidad: obtener el id y role del payload del token.
8. Validación de sesión: confirmar que la sesión (sid) sigue activa en la base de datos.
9. Verificación de rol: comprobar que el role del usuario está en la lista de roles permitidos para esa ruta.
10. Autorizar o rechazar: si pasa, llamar a next(); si falla, responder 401 o 403 según el caso.

4.2 Diferencia entre 401 y 403

Código HTTP	Significado y cuándo usarlo
401 Unauthorized	El usuario no está autenticado. No se proporcionó token, el token es inválido, o la sesión fue revocada. El cliente debe iniciar sesión.
403 Forbidden	El usuario está autenticado pero no tiene permiso para ese recurso. El rol no es suficiente. El cliente no debe reintentar (mismas credenciales no ayudarán).

4.3 Principio de Mínimo Privilegio

El principio de mínimo privilegio (Principle of Least Privilege) establece que cada usuario, proceso o sistema debe tener únicamente los permisos estrictamente necesarios para realizar su función. En la práctica, esto significa: diseñar primero los roles y sus permisos antes de escribir código, negar por defecto (todo está bloqueado hasta que explícitamente se permite) y revisar periódicamente los permisos para eliminar los que ya no son necesarios.

PARTE II — ENTREGABLES

5. Rutas Protegidas por Rol (Implementación)

El sistema implementa protección de rutas mediante dos middlewares propios ubicados en `src/middlewares/`. A continuación se describe el código implementado, su funcionamiento y la cobertura de rutas.

5.1 Middleware de Autenticación (`authMiddleware.js`)

Este middleware valida el access token JWT y verifica en la base de datos PostgreSQL que la sesión asociada siga activa. Si cualquiera de las dos validaciones falla, la petición es rechazada antes de llegar al controlador.

```
const jwt = require('jsonwebtoken');
const { JWT_SECRET } = require('../config/authTokens');
const { assertSessionActive } = require('../services/sessionService');

async function authenticate(req, res, next) {
  const authHeader = req.header('Authorization');

  if (!authHeader)
    return res.status(401).json({ msg: 'Acceso denegado. No hay token.' });

  const token = authHeader.startsWith('Bearer ')
    ? authHeader.slice(7) : authHeader;

  try {
    const decoded = jwt.verify(token, JWT_SECRET);

    if (!decoded.sid)
      return res.status(401).json({ msg: 'Token sin sesión.' });

    const active = await assertSessionActive(decoded.sid, decoded.id);
    if (!active)
      return res.status(401).json({ msg: 'Sesión revocada o expirada.' });

    req.user = decoded; // { id, role, zone_id, sid }
    req.sessionId = decoded.sid;
    next();
  } catch {
    res.status(401).json({ msg: 'Token no válido o expirado.' });
  }
}
```

El middleware sigue este flujo de decisión: extrae el token del header Authorization, intenta verificarlo con la clave secreta JWT_SECRET, verifica que el token contenga un campo sid (session ID), consulta en base de datos si esa sesión sigue activa, y finalmente inyecta el payload en req.user para que los middlewares posteriores lo usen.

5.2 Middleware de Autorización por Rol (authorizeRoles.js)

Este middleware es una función de orden superior (higher-order function): recibe una lista de roles permitidos y devuelve un middleware Express. Se encadena siempre después de authenticate.

```
function authorizeRoles(...allowedRoles) {
  const allowed = new Set(allowedRoles);

  return (req, res, next) => {
    if (!req.user)
      return res.status(401).json({ msg: 'No autenticado.' });

    const role = req.user.role;
    if (!role || !allowed.has(role))
      return res.status(403).json({ msg: 'No tienes permisos para esta acción.' });

    next();
  };
}
```

El uso de Set en lugar de Array.includes() mejora el rendimiento en listas largas de roles y hace el código más legible. La distinción 401/403 es semánticamente correcta: 401 si no hay usuario autenticado (uso incorrecto del middleware), 403 si el usuario está autenticado pero no tiene el rol necesario.

5.3 Mapa Completo de Rutas y Protecciones

Método	Ruta	Protección	Roles permitidos
POST	/api/auth/register	Pública	—
POST	/api/auth/login	Pública	—
POST	/api/auth/recover	Pública	—
POST	/api/auth/reset-password	Pública	—

POST	/api/auth/request-otp	Pública	—
POST	/api/auth/verify-otp	Pública	—
POST	/api/auth/reset-password-otp	Pública	—
POST	/api/auth/refresh	Pública	—
POST	/api/auth/logout	authenticate	Todos los autenticados
POST	/api/auth/logout-all	authenticate	Todos los autenticados
GET	/api/auth/sessions	authenticate	Todos los autenticados
DELETE	/api/auth/sessions/:id	authenticate	Todos los autenticados
GET	/api/auth/me	authenticate	Todos los autenticados
GET	/api/admin/users	authenticate + authorizeRoles	admin
POST	/api/auditoria/registrar	authenticate + authorizeRoles	guard, admin
GET	/api/health	Pública	—

6. Evidencia de Bloqueo de Accesos Indebidos

Esta sección documenta los comportamientos de seguridad esperados del sistema, verificables mediante pruebas con Postman u otras herramientas HTTP. Cada caso de prueba corresponde a un escenario de intento de acceso no autorizado y la respuesta correcta del servidor.

6.1 Intento sin Token (401 Unauthorized)

Cuando una petición llega a una ruta protegida sin el header Authorization, el middleware authenticate detecta la ausencia del token y rechaza inmediatamente la petición.

Campo	Valor
Petición	GET /api/admin/users
Headers	(ninguno relacionado con autorización)
Respuesta esperada	HTTP 401
Cuerpo de respuesta	{ "msg": "Acceso denegado. No hay token." }
Middleware que actúa	authenticate (primera verificación)
Lógica ejecutada	if (!authHeader) → return res.status(401)

Campo	Valor
Petición	GET /api/admin/users con token de usuario con rol 'driver'
Header	Authorization: Bearer <token_driver_válido>
Respuesta esperada	HTTP 403
Cuerpo de respuesta	{ "msg": "No tienes permisos para esta acción." }
Middleware que actúa	authorizeRoles('admin')

Lógica ejecutada

allowed = new Set(['admin']); !allowed.has('driver') → 403

6.5 Acceso Legítimo con Rol Correcto (200/201 OK)

Para confirmar que el sistema no solo bloquea accesos indebidos, sino que también permite los correctos, se verifica el comportamiento con credenciales válidas y el rol adecuado.

Campo	Valor
Petición	GET /api/admin/users con token de usuario con rol 'admin'
Header	Authorization: Bearer <token_admin_válido>
Respuesta esperada	HTTP 200
Cuerpo de respuesta	{ "users": [...], cada usuario sin campo password_hash }
Validación adicional	La respuesta no expone password_hash ni datos sensibles de sesión.

7. Tests de Seguridad

Los tests de seguridad automatizados se encuentran en el archivo tests/roles.test.js. Se implementaron con Jest como framework de testing y Supertest como cliente HTTP para realizar peticiones HTTP al servidor Express sin necesidad de un servidor en ejecución.

7.1 Configuración del Entorno de Testing

Los tests de RBAC requieren una conexión real a PostgreSQL porque incluyen validación de sesión activa. El archivo jest.config.js configura el entorno y los tests se condicionan a la presencia de la variable DB_NAME en el archivo .env:

```
const runRolesTests = Boolean(process.env.DB_NAME);
(runRolesTests ? describe : describe.skip)('RBAC y rutas protegidas',
() => {
  // tests aquí...
});
```

7.2 Preparación de Datos (beforeAll / afterAll)

El bloque beforeAll crea los usuarios de prueba directamente en la base de datos con contraseñas hashadas con bcrypt, realiza el login para obtener los tokens de cada rol, y aplica las migraciones necesarias. El bloque afterAll elimina todos los datos de prueba y cierra el pool de conexiones.

7.3 Casos de Prueba Implementados

Los tests cubren dos rutas protegidas (GET /api/admin/users y POST /api/auditoria/registrar) y verifican el comportamiento para cada combinación de autenticación/rol:

Test	Ruta	Escenario	Respuesta esperada
Sin token	GET /api/admin/users	Petición sin header Authorization	401
Rol driver	GET /api/admin/users	Token válido pero rol driver	403
Rol admin + sin hash	GET /api/admin/users	Token admin — verifica que no hay password_hash en respuesta	200
Sin token	POST /api/auditoria/registrar	Petición sin header Authorization	401
Rol driver	POST /api/auditoria/registrar	Token válido pero rol driver	403
Rol guard	POST /api/auditoria/registrar	Token guard — acceso legítimo	201
Rol admin	POST /api/auditoria/registrar	Token admin — acceso legítimo	201

7.4 Código de los Tests

A continuación se muestra la estructura de los tests más relevantes:

```
describe('GET /api/admin/users', () => {

  it('401 sin token', async () => {
    const res = await request(app).get('/api/admin/users');
    expect(res.status).toBe(401);
  });

  it('403 con rol driver', async () => {
    const res = await request(app)
      .get('/api/admin/users')
      .set('Authorization', `Bearer ${tokenDriver}`);
    expect(res.status).toBe(403);
    expect(res.body.msg).toBeTruthy();
  });
});
```

```

it('200 con rol admin y lista sin password_hash', async () => {
  const res = await request(app)
    .get('/api/admin/users')
    .set('Authorization', `Bearer ${tokenAdmin}`);
  expect(res.status).toBe(200);
  expect(Array.isArray(res.body.users)).toBe(true);
  expect(res.body.users[0]).not.toHaveProperty('password_hash');
});
});

```

7.5 Ejecución de los Tests

Para ejecutar los tests es necesario tener configurado el archivo .env con las variables de conexión a PostgreSQL:

```

DB_HOST=localhost
DB_PORT=5432
DB_NAME=nombre_base_de_datos
DB_USER=usuario
DB_PASS=contraseña
JWT_SECRET=clave_secreta_jwt

```

Comando para ejecutar los tests:

```
npm test
```

Comando para ver cobertura de código:

```
npm test -- --coverage
```

8. Pruebas con Postman — Guía de Evidencia

Postman es una herramienta de prueba de APIs que permite enviar peticiones HTTP y verificar las respuestas. A continuación se documenta el conjunto de pruebas recomendadas para generar evidencia visual del sistema de protección de rutas.

8.1 Configuración Inicial en Postman

Antes de ejecutar las pruebas, crear un Environment en Postman con las siguientes variables:

Variable	Valor de ejemplo
base_url	http://localhost:3000

token_admin	(se llenará después del login de admin)
token_guard	(se llenará después del login de guard)
token_driver	(se llenará después del login de driver)

8.2 Prueba 1 — Obtener Token Admin

Esta prueba obtiene el access token de un usuario con rol admin que será utilizado en las pruebas de acceso autorizado.

Campo	Valor
Método	POST
URL	{{base_url}}/api/auth/login
Body (JSON)	{ "phone": "5611111111", "password": "testpass123" }
Resultado esperado	HTTP 200 con campo accessToken en el cuerpo
Acción post-respuesta	Copiar el valor de accessToken y guardarlo en la variable token_admin
Evidencia a capturar	Screenshot de la respuesta 200 con el token

8.3 Prueba 2 — Acceso sin Token a Ruta Protegida (401)

Verifica que el sistema rechaza correctamente cualquier petición a rutas protegidas cuando no se proporciona un token de autenticación.

Campo	Valor
Método	GET
URL	{{base_url}}/api/admin/users
Headers	(no incluir Authorization)
Resultado esperado	HTTP 401
Cuerpo esperado	{ "msg": "Acceso denegado. No hay token." }
Evidencia a capturar	Screenshot mostrando el código 401 y el mensaje de error

8.4 Prueba 3 — Token Inválido (401)

Verifica que el sistema detecta y rechaza tokens manipulados o firmados con un secreto diferente.

Campo	Valor
Método	GET

URL	{{base_url}}/api/admin/users
Header Authorization	Bearer esto.no.es.un.token.valido
Resultado esperado	HTTP 401
Cuerpo esperado	{ "msg": "Token no válido o expirado." }
Evidencia a capturar	Screenshot mostrando el rechazo del token inválido

8.5 Prueba 4 — Rol Incorrecto (403)

Este es el test más importante para demostrar el control de acceso por rol. Un usuario autenticado con rol driver intenta acceder a un recurso de administración.

Paso 1: Obtener token con rol driver (misma petición que la Prueba 1, pero con las credenciales del driver). Paso 2: Usar ese token para acceder a la ruta de admin:

Campo	Valor
Método	GET
URL	{{base_url}}/api/admin/users
Header Authorization	Bearer {{token_driver}}
Resultado esperado	HTTP 403
Cuerpo esperado	{ "msg": "No tienes permisos para esta acción." }
Evidencia a capturar	Screenshot mostrando código 403 con token válido pero rol insuficiente

8.6 Prueba 5 — Acceso Autorizado (200)

Verifica que el sistema permite el acceso cuando el usuario tiene el rol correcto, y que la respuesta no expone datos sensibles como contraseñas hasheadas.

Campo	Valor
Método	GET
URL	{{base_url}}/api/admin/users
Header Authorization	Bearer {{token_admin}}
Resultado esperado	HTTP 200 con array de usuarios
Verificación adicional	Confirmar que ningún objeto del array contiene el campo password_hash
Evidencia a capturar	Screenshot de la respuesta 200 y el array de usuarios sin datos sensibles

8.7 Prueba 6 — Auditoria con Rol Guard (201)

Verifica que el rol guard puede realizar el registro de auditoría pero no acceder a administración.

Campo	Valor
Método	POST
URL	{{base_url}}/api/auditoria/registrar
Header Authorization	Bearer {{token_guard}}
Body (JSON)	{ "quien_entra": "demo-qr", "quien_autoriza": "guardia", "destino": "Calle Demo 1" }
Resultado esperado	HTTP 201 con mensaje de confirmación
Evidencia a capturar	Screenshot de la respuesta 201

8.8 Prueba 7 — Sesión Revocada (401 post-logout)

Esta prueba más avanzada demuestra la validación de sesión en base de datos. Requiere dos pasos:

11. Hacer POST /api/auth/logout con el token de admin. Respuesta esperada: 200.

12. Inmediatamente después, intentar GET /api/admin/users con el mismo token. Respuesta esperada: 401 con mensaje 'Sesión revocada o expirada.'

Campo	Valor
Evidencia a capturar	Dos screenshots: uno del logout exitoso y otro del rechazo posterior
Importancia	Demuestra que la revocación de sesión funciona aunque el token JWT siga siendo criptográficamente válido

8.9 Resumen de Pruebas y Resultados Esperados

#	Prueba	HTTP esperado	Comportamiento verificado
1	Login admin	200	Obtener token válido
2	Sin token → admin/users	401	Rechazo sin autenticación

3	Token inválido → admin/users	401	Rechazo de token manipulado
4	Token driver → admin/users	403	Rechazo por rol insuficiente
5	Token admin → admin/users	200	Acceso autorizado, sin password_hash
6	Token driver → auditoria/regar	403	Rechazo por rol insuficiente
7	Token guard → auditoria/regar	201	Acceso autorizado para guard
8	Token admin → auditoria/regar	201	Acceso autorizado para admin
9	Logout + mismo token	401	Sesión revocada en BD